Experiment 9: Use pytest to test python application

1. Objective

- Understand unit testing concepts
- Write test cases using PyTest
- Execute and analyze test results
- Handle edge cases and exceptions
- Use parameterized testing

2. Prerequisites

- Python
- Pytest

3. Installation

Install pytest using pip:

```
pip install pytest
```

Verify it :

```
pytest –version
```

4. Create a Python module with basic mathematical operations and write PyTest test cases to validate:

- Correct outputs
- Edge cases
- Exception handling

Step 1: Create Project PythonDemo and add file pythondemo.py

```
def add(a, b):

    return a + b


def subtract(a, b):

    return a - b
```

Experiment 9: Use pytest to test python application

```
def multiply(a, b):
    return a * b


def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b


def is_even(num):
    return num % 2 == 0
```

Step 2: Create a test file test_pythondemo.py

```
import pytest
import pythondemo
# Passing test
def test_add():
    assert pythondemo.add(2, 3) == 5
# Failing test
def test_multiply_fail():
    assert pythondemo.multiply(4, 3) == 11
# Exception test
def test_divide_by_zero():
    with pytest.raises(ValueError):
        pythondemo.divide(10, 0)
```

Experiment 9: Use pytest to test python application

```
# Logical test

def test_is_even():

    assert pythondemo.is_even(4) is True

    assert pythondemo.is_even(5) is False


# Parametrized test

@pytest.mark.parametrize("a,b,result", [

    (1,2,3),

    (2,2,4),

    (5,5,11)   # failing case

])

def test_add_param(a,b,result):

    assert pythondemo.add(a,b) == result
```

5. Execution

Open terminal in that folder
pytest –v

```
D:\Pytest_demo>notepad test_pythondemo.py

D:\Pytest_demo>pytest -v
==================================== test session starts ====================================
platform win32 -- Python 3.14.2, pytest-9.0.1, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: D:\Pytest_demo
collected 13 items

test_pythondemo.py::test_add PASSED                                                  [  7%]
test_pythondemo.py::test_subtract PASSED                                             [ 15%]
test_pythondemo.py::test_multiply_fail FAILED                                        [ 23%]
test_pythondemo.py::test_divide PASSED                                               [ 30%]
test_pythondemo.py::test_divide_by_zero PASSED                                       [ 38%]
test_pythondemo.py::test_is_even_fail FAILED                                         [ 46%]
test_pythondemo.py::test_type_fail FAILED                                            [ 53%]
test_pythondemo.py::test_add_param[1-2-3] PASSED                                     [ 61%]
test_pythondemo.py::test_add_param[10-20-30] PASSED                                  [ 69%]
test_pythondemo.py::test_add_param[0-5-5] PASSED                                     [ 76%]
test_pythondemo.py::test_add_param_fail[2-2-4] PASSED                                [ 84%]
test_pythondemo.py::test_add_param_fail[3-3-6] PASSED                                [ 92%]
test_pythondemo.py::test_add_param_fail[5-5-11] FAILED                               [100%]

========================================= FAILURES ==========================================
___________________________________ test_multiply_fail ___________________________________

    def test_multiply_fail():
>       assert pythondemo.multiply(4, 3) == 11   # actual is 12
```

Experiment 9: Use pytest to test python application

Task for you:

Task 1:

Add a function:

```
def factorial(n):
```

Write test cases.

Task 2:

Write test cases for:

Negative numbers

Large inputs